

Java文件处理

文件类

Java.io.File就是文件类，可以通过该类操作文件；

File("path")创建File对象

```
File f1 = new File("d://2.txt"); //提供文件的完整路径
File f2 = new File("d://abc");
// 这个文件路径存在或者不存在不重要，在这里只是负责“创建”File
// 这行代码并不会对文件进行任何操作
```

mkdir()、createNewFile() 创建文件

```
// 创建文件
f1.createNewFile(f1);
// 创建文件夹
f2.mkdir(f2);
// 注： 这两个方法的返回值都是布尔类型，创建成功返回true，创建失败返回false（如果文件或者路径已经存在就会失败）
// 这两个方法失败就只是返回false，不会抛出异常
```

File("parent", "child")创建File对象

```
// 通过“拼接”的方式创建File对象
File f3 = new File("d://abc", "1.jpg"); // 等价于"d://abc/1.jpg"
File f4 = new File(f2, "2.gif"); // 等价于"d://abc/2.gif"
```

delete() 删除文件

```
f1.delete();
// 注意：删完了之后无法恢复
```

获取文件长度

```
File f = new File("d://abc//成绩.png");  
long size = f.length(); // 单位 字节  
System.out.println(size);
```

获取文件名、上级文件夹名、路径、绝对路径

```
File f = new File("d://abc//成绩.png");  
  
String s1 = f.getName(); // 成绩.png  
String s2 = f.getParent(); // d://abc  
String s3 = f.getPath(); // d://abc/成绩.png  
String s4 = f.getAbsolutePath(); // d://abc/成绩.png  
String s5 = f.getCanonicalPath(); // D://abc/成绩.png
```

文件的基本特征

读、写、执行权限；是否隐藏；是否为文件；是否为目录；路径是否存在

```
boolean b1 = f.canRead(); //是否可读？  
boolean b2 = f.canWrite(); //是否可写？  
boolean b3 = f.canExecute(); //是否可执行？  
boolean b4 = f.isHidden(); //是否隐藏？  
boolean b5 = f.isDirectory(); //是否为一个目录？  
boolean b6 = f.isFile(); //是否为一个文件？  
boolean b7 = f.exists(); //文件或目录是否存在？
```

获取目录下的所有目录和文件

```
File d = new File("c://program files");  
File[] fs = d.listFiles();
```

流

流的分类

输入流、输出流

从别的地方往程序内存里进的就是输入流；

从程序内存往外输出数据的就是输出流；

字节流、字符流

	输入	输出
字节流	InputStream	OutputStream
字符流	Reader	Writer

原始流、处理流（装饰流、节点流）

原始流

例：读取文件->写入另一个文件

```
// 场景：从文件中读取内容，并输出到另一个文件中
public static void main(String[] args) throws Exception{
    File input = new File("d://成绩.png"); //源文件
    File output = new File("c://成绩.png");//目标文件

    // 此处的流：FileInputStream、FileOutputStream就是原始流
    // 区别于处理流的特征：原始流的参数 不是另一个流！
    FileInputStream fis = new FileInputStream(input); //源文件读入内存
    FileOutputStream fos = new FileOutputStream(output); //从内存向目标文件写出

    byte[] b = new byte[1024]; //建立1kB的数组用于读取和写出数据（长度根据实际设定）
```

```

// read并非一定要读取1024个byte
// read返回值为成功读取的byte长度
// read读取失败时会返回-1
int len = fis.read(b); //将源文件中的1kB数据读取到b数组中，len是读取长度
while(len != -1){
    //写入目标文件
    fos.write(b, 0, len); //写出b数组[0,len)的内容
    //读取下一个1024
    len = fis.read(b);
}
//关闭原始流
fis.close();
fos.close();
}

```

处理流（装饰流、节点流）

通过包装基本流，获取一些额外功能的流；

#####

FileWriter

```

public static void main(String[] args) throws Exception {
    List<String> list = new ArrayList<>();
    list.add("09/25-10/02");
    list.add("10/11-10/14");
    list.add("10/20-10/29");

    FileWriter fw = new FileWriter("d://data.txt");
    for(String s : list){
        fw.write(s);
        fw.write("\n");
    }
}

```

```
fw.close();

}
```

BufferedReader

例如：要求按行读取，此时再通过byte数组和原始流配合，就面临着不知道byte数组应当设置多大合适的问题；因为一行的长度是不固定的；

```
public static void main(String[] args) throws Exception{
    FileReader fr = new FileReader("d://info.txt");
    // 使用br装饰了fr，操作br即等同于操作fr
    // 区别于原始流的特征：处理流的参数本身就是一个流
    // 这相当于在一个流外面又套了一层流...这种写代码的模式也被称为：装饰模式
    // 所以处理流也被成为装饰流

    BufferedReader br = new BufferedReader(fr);

    // BufferedReader可以直接读取一行
    // 原始流是没有这种功能的
    String line = br.readLine();
    while(line != null){
        //本案例打印，其他根据实际情况
        System.out.println(line);
        line = br.readLine();
    }

    //注意顺序
    br.close(); // 先释放处理流
    fr.close(); // 最后释放原始流
}
```

异常的处理

```
// 简洁的处理方式
```

```

public static void main(String[] args){
    File input = new File("d://成绩.png");
    File output = new File("c://成绩.png");

    // try-with-resource (自动关闭流)
    // 相当于fis、fos只在try...catch...的周期里存活
    try(FileInputStream fis = new FileInputStream(input);
        FileOutputStream fos = new FileOutputStream(output);){
        byte[] b = new byte[1024];
        int len = fis.read(b);
        while(len != -1){
            fos.write(b, 0, len);
            len = fis.read(b);
        }
    } catch (Exception e){
        e.printStackTrace();
    }
}

```

如果不适用try with resource, 那么异常处理会变成一个很冗长的流程...参见下列代码:

```

public static void main(String[] args){
    File input = new File("d://成绩.png");
    File output = new File("c://成绩.png");
    FileInputStream fis = null;
    FileOutputStream fos = null;
    try {
        fis = new FileInputStream(input);
        fos = new FileOutputStream(output);
        byte[] b = new byte[1024];

        int len = fis.read(b);
        while(len != -1){
            fos.write(b, 0, len);
            len = fis.read(b);
        }
    }catch (Exception e){
        // todo
    }
}

```

```

    }finally {
        try {
            if(fis != null){ // 防止new FileInputStream的时候就失败
                fis.close();
            }

        } catch (IOException e) {
            // todo
        }
        try { // 为什么要分成两个try关闭? 因为要防止第一个close失败, 跳过对第二个流的处理
            if(fos != null){
                fos.close();
            }
        } catch (IOException e) {
            // todo
        }
    }
}

```

序列化

直接将内存对象，转化为二进制数据的形式，序列化需要注意两个事：

1. 对象必须实现Serializable才能被序列化
2. 必须包含一个long类型的UID

```

// 例：
public class Employee implements Serializable {
    public static final long serialVersionUID = 849824901280934810L;
    // ....
}

```

对象的序列化与反序列化：

序列化： 对象->文件

反序列化： 文件->对象

// 序列化

```
Employee e1 = new Employee(1004,"xxx", new Date());  
Employee e2 = new Employee(1005,"zzz", new Date());  
FileOutputStream fos = new FileOutputStream("d://emp.haha");  
ObjectOutputStream oos = new ObjectOutputStream(fos);
```

```
List<Employee> list = new ArrayList<>();  
list.add(e1);  
list.add(e2);
```

```
oos.writeObject(list);  
oos.flush();  
fos.close();
```

// 反序列化

```
FileInputStream fis = new FileInputStream("d://emp.haha");  
ObjectInputStream ois = new ObjectInputStream(fis);  
  
List<Employee> list = (List<Employee>) ois.readObject();  
for(Employee e : list){  
    System.out.println(e);  
}  
fis.close();
```

